

★ KUBERNETES YAML EXPLAINED ★

By Tushar Sherkar

1. What is Kubernetes YAML?

- YAML (Yet Another Markup Language) is used to define Kubernetes resources.
- We declare the desired state in YAML and Kubernetes makes it happen.
- YAML files are used with kubectl to create, update and manage resources.

2. Basic Structure of YAML

```
apiVersion: <group>/<version>
kind: <Resource Kind>
metadata:
  name: <name-of-resource>
  labels:
    key: value
spec:
  # desired state of the resource
```

apiVersion → API version of Kubernetes

kind → Type of resource (Pod, Service, etc.)

metadata → Data about the resource

spec → Desired state / configuration

3. Important Fields

- **apiVersion**: Version of the K8s API
- **kind**: Type of Kubernetes object
- **metadata**: Name, labels, annotations, etc.
- **spec**: Desired state of the object
- **status**: Current state (added by K8s)

- 💡 Always indent using 2 spaces
- YAML is case-sensitive
- Use "--" for lists
- # is used for comments

4. YAML Example - Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: myapp
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

→ K8s API version
→ Type of resource
→ Name of the Pod
→ Label (key: value)
→ List of containers
→ Name of container
→ Docker image
→ Container port

5. YAML Example - Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

→ Deployment API version
→ Kind is Deployment
→ Name of Deployment
→ Label for Deployment
→ No. of Pod replicas
→ Selects Pods with this label
→ Template for Pods
→ Labels for Pods
→ Container definition
→ Container image
→ Port exposed

How it works?

Deployment creates ReplicaSet, ReplicaSet creates Pods. If a Pod fails, new Pod is created to maintain replicas. 😊

6. YAML Example - Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

→ Service API version
→ Kind is Service
→ Service name
→ Service type
→ Selects Pods
→ Port exposed by Service
→ Port on Pod

Service Types:

- **ClusterIP** - Internal access (default)
- **NodePort** - Access using Node IP + Port
- **LoadBalancer** - Exposes via cloud LB
- **ExternalName** - Maps to external DNS

7. YAML Example - ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: "MyApp"
  APP_ENV: "Production"
  LOG_LEVEL: "INFO"
```

Use in Pod:

```
envFrom:
  - configMapRef:
      name: app-config
```

→ Inject all key-values as environment variables

8. YAML Example - Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: dXNlcg==
  password: cGFzc3dvcrQ=
```

Data is Base64 encoded.

Use in Pod as environment variable or volume mount.

9. Common Kubernetes Resources & APIs

Resource (kind)	API Version
Pod	v1
Service	v1
ConfigMap	v1
Secret	v1
Deployment	apps/v1
ReplicaSet	apps/v1
StatefulSet	apps/v1
DaemonSet	apps/v1
Job	batch/v1
CronJob	batch/v1
Ingress	networking.k8s.io/v1

10. Useful kubectl Commands

- **kubectl apply -f file.yaml** → Create/Update resource
- **kubectl get all** → List all resources
- **kubectl get pods -o wide** → Pod details
- **kubectl describe pod <pod-name>** → Detailed info
- **kubectl logs <pod-name>** → View logs
- **kubectl delete -f file.yaml** → Delete resource
- **kubectl exec -it <pod> -- /bin/bash** → Shell into Pod

Tips

- 💡 Use meaningful names.
- ✓ Use labels to organize resources.
- ✓ Keep YAML files in version control (Git).
- ✓ Test changes in lower environment first.



★ Kubernetes YAML is all about declaring the desired state. Kubernetes takes care of the rest! ★

- Tushar Sherkar 😊